CS230 Getting Started

Abhijeet Shenoi and Lucio Mwinmaarong Dery

April 2018

1 Unix Walkthrough

The terminal is the original set based interface that was the fad back when computers had RAM in the kilobytes, and monochrome monitors. In other words, it's the text based approach to doing things that you're more used to doing with your mouse.

When you fire up your terminal, it's currently in a working directory. This is usually your home directory. But if you ever lose track, you can always type out the command `pwd` - print working directory.

Now that we're here, you want to know what files are in your directory. This is done by the command `ls`. `ls` will print all the files and directories, in your current working directory.

To navigate into a directory which is in the current working directory, type in `cd` followed by the name of your directory. For example, if in your current directory, there's a folder called deep_learning, you'd type in `cd deep_learning`.

Another pro-tip is the use of `.` or `..` .

Every folder has two 'files' called . and .. , which help you quickly refer to either the current directory, or the parent directory. So to quickly move up one directory, you can type in `cd ..`

There are a bunch of other commands that help you navigate the terminal and you can check them out here and here

Note that even though these are mac tutorials, this is how the terminal works in Linux as well.

2 Using Pip and Virtual Environments

Guide to pip:

You want to install pip by going to this link. Most of you should have it already installed by default though. To verify, type in `pip version` into your terminal. You should see the installed version of pip.

pip is a python package manager. It lets you install all the bells and whistles that makes python fancy, such as numpy, tensorflow, h5py and everything else.

To install a package, type in pip install numpy, or any other package name that you need. A particularly useful package that you might want is called virtualenv.

Virtualenv lets you insulate different 'environments' from each other. Say you have a tensorflow version 1.4.1, but you have an assignment written in version 0.4. Do you replace your system version with 0.4?

There's an easier way to do this. Setup a virtual environment in your python folder. Activate the environment. Once you do this, your python is effectively that which is setup in this environment of yours. Any installations that you make will only affect your environment. Once you deactivate the environment, you go back to your system version of python, almost as if you never installed anything. Let's walkthrough setting up a virtual environment.

First, pip install virtualenv.

Then move to the folder where you want to install your virtual environment. Once there, type in virtualenv ¡name of your environment¿. To setup an environment called cs230, type in virtualenv cs230.

Once the installation takes place, to activate your environment, type in source ¡environment name¿/bin/activate For example, type source cs230/bin/activate You can now pip install packages, completely independent of your system installation. Any python scripts you run while the environment is active (as you will see in your terminal) will use only the installed packages.

Once you're done, you can deactivate the environment by typing in deactivate.

Note that if you reactivate your environment then you still have all your previously installed packages.

This is great especially if you have multiple courses operating with different versions of packages.

Check this out for more information: https://packaging.python.org/guides/installing-using-pip-and-virtualenv/

3 Relevant Git Information

3.1 What is Git

Git is a version control tool. Let's unpack this a bit. Imagine you are working on writing a book. You might have different versions of the book, maybe to capture different endings, and those versions might be in different stages of development. You might want to be able to save new changes, go back to older versions of the book to remember things or delete all new changes and start again from an earlier stage. Git allows us to have this control, but for our code.

All this is done locally on your computer. To safeguard your data (in case your machine gets damaged), and also to allow you to collaborate with other people, you can host your code on Github. Bitbucket is another platform you can use to host your code. In this tutorial we will be using Github. Remember - Git is the version control tool. Github and Bitbucket are just hosting websites

and they allow you to push code from one computer and pull it to another computer.

3.2 Installing Git

To install on machine, please follow the instructions at this link. Please pass by Office Hours if you run into issues - they tend to be environment dependent.

3.3 Getting Started

The first step is to create a Github account. Please signup at this link.

After signing up, you will be asked to chose between public (free) repository / project and private repository (currently $7/month). Public repos are visible to everyone on the internet, so if you have a project you want to keep secret, you can go for the private option.

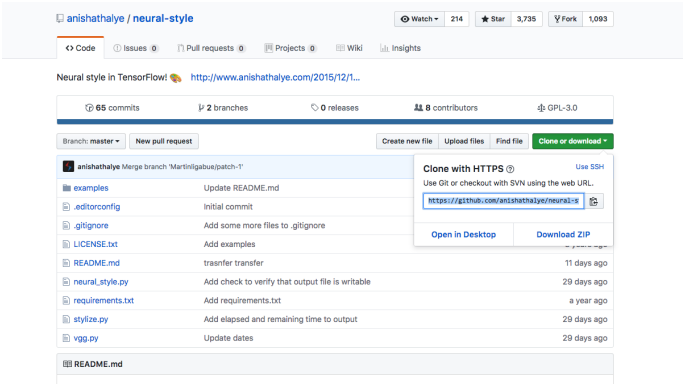
Starting a fresh Github project from the Github website

After setting up your Github acccount, you can start a new project / repository. Follow the instructions here to create a repository from your Github account.

Downloading a pre-existing Github project

The `git clone` command is used to download existing github repository. Let's say you want to clone this Neural Style Project.

1. Copy the link to the repository shown in the clone/download button on github



2. Open up a terminal window and navigate to the folder where you want to create the project. Use the cd command. Clone the repository



Starting a fresh Github project from your laptop.

1. Open up a terminal window

2. Navigate to the folder where you want to create the project. Use the cd command.

```
DN0a24e723:~ luciodery$ cd Desktop/Projects/git_tutorial/
```

3. Initialize the git repository. Use `git init`

```
DN0a24e723:git_tutorial luciodery$ git init
Initialized empty Git repository in /Users/luciodery/Desktop/Projects/git_tutorial/.git/
DN0a24e723:git_tutorial luciodery$
```

4. The typical workflow is to create files/folders and make changes, save your changes.

   (a) Do work in the folder. As an example, we create a file called README.txt using the echo command. You can create folders and files, and populate them with code, etc.

```
DN0a24e723:git_tutorial luciodery$ echo "GIT TUTORIAL REPO" >> README.txt
DN0a24e723:git_tutorial luciodery$ ls
README.txt
DN0a24e723:git_tutorial luciodery$ cat README.txt
GIT TUTORIAL REPO
DN0a24e723:git_tutorial luciodery$
```

   (b) Use `git status` to find out the state of your repo.

```
DN0a24e723:git_tutorial luciodery$ git status
On branch master

Initial commit

Untracked files:
  (use "git add <file>..." to include in what will be committed)

        README.txt

nothing added to commit but untracked files present (use "git add" to track)
DN0a24e723:git_tutorial luciodery$
```

   The main thing to get out of the prompt is that we have created a file, README.txt but git does not know about this file so it is untracked.

   (c) Add the file to the git repo so that it is tracked. Use `git add`
   Save your changes using `git commit`. You can use the -m flag to store a helfpul message about your commit.

```
DN0a24e723:git_tutorial luciodery$ git add README.txt
DN0a24e723:git_tutorial luciodery$ git commit -m "Added README.txt file to repo"
[master (root-commit) acaeeba] Added README.txt file to repo
 1 file changed, 1 insertion(+)
 create mode 100644 README.txt
```

   To see a summary of your commits (saves) use `git log`.

```
DN0a24e723:git_tutorial luciodery$ git log
commit acaeeba30e5d09ec68c1067d02b27dfff09b039c
Author: Lucio Dery <ldery@stanford.edu>
Date:   Wed Apr 25 17:49:24 2018 -0700

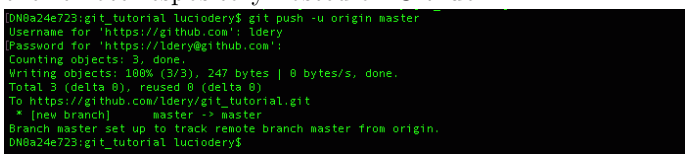
    Added README.txt file to repo
DN0a24e723:git_tutorial luciodery$
```

5. Upload your repo to github.To do this, first create a corresponding repo on the Github website. Follow the instructions linked above in the "Starting a fresh Github project from the Github website".
   Use the `git remote add origin` command to link your local repository to the repository you created online.

```
DN0a24e723:git_tutorial luciodery$ git remote add origin https://github.com/ldery/git_tutorial.git
```

   Use `git push` to push the changes you have made and new files added to

the remote respository hosted on Github.



The above instructions link the local repo to the repo created on the github website. It also creates a branch (a version of the book - going back to the book analogy) called master to which the current state of the project is uploaded. For more information about branches, see the section below.

3.4 Getting Familiar with Git

Git is very powerful and can save you a world of hurt if you use it properly and consistently. Here are a couple of links here and here of useful commands to know. The second link provides a lot of information on how to perform commands you would ordinarily do on a terminal in git, directly on Github using the provided UI : the entry barrier to version control effectively is much less if you take this route. StackOverflow is also a great resource for looking up questions relating to git. The link provided has is also a practical guide to Git with common questions and answers provided